MASTER STUDY HANDBOOK AND LECTURE COMPENDIUM

Artificial Intelligence: Foundations, Principles and Search

Synthesized from Course Lectures by Khaula Qadeer, Stuart Russell, and Cesare Tinelli

Recommended Video: AI Foundations and Turing Test (YouTube Search Query: `Artificial Intelligence History and Foundations Russell Norvig MIT`).

1. The Fundamental Philosophy and Scope of Artificial Intelligence

For thousands of years, philosophers and scientists have sought to understand human cognition: how can a physical brain, composed of biological matter, perceive, understand, predict, and manipulate an external world that is infinitely larger and more complex than itself? Cognitive science and psychology analyze how natural minds operate. Artificial Intelligence (AI) advances this quest further: it strives not only to explain intelligence, but to computationally construct intelligent entities capable of solving complex tasks autonomously.

2. The Four Historical Perspectives on Artificial Intelligence

Historically, researchers have conceptualized AI across two intersecting dimensions: **Thought Processes vs Behavior**, and **Human Fidelity vs Rationality**. This yields four distinct paradigms:

CORE CONCEPTUAL MATRIX: THE FOUR DEFINITIONS OF AI

- **Thinking Humanly (Cognitive Modeling):** Validated through psychological experiments and cognitive science. The program must execute intermediate reasoning steps that mirror the human brain (for example, Newell and Simon's General Problem Solver, 1961).
- **Acting Humanly (The Turing Test Approach):** Proposed by Alan Turing in 1950. Operational intelligence is demonstrated if an interrogator in a separate room cannot distinguish between responses from a human and a computer. Requires Natural Language Processing, Knowledge Representation, Automated Reasoning, and Machine Learning. The Total Turing Test further requires Computer Vision and Robotics.
- **Thinking Rationally (The Laws of Thought):** Rooted in Aristotle's syllogisms (for example: All humans are mortal; Socrates is a human; therefore, Socrates is mortal). Uses formal symbolic logic to govern irrefutable inference. Limitations: real-world knowledge is rarely clean and deterministic, and logical theorem proving faces computational intractability.
- **Acting Rationally (The Rational Agent Approach):** Focuses on creating agents that act to achieve the best outcome, or the best expected outcome when operating under uncertainty. This serves as the modern unifying foundation of the field.

3. The Imitation Game, Chatbots, and the Aviation Analogy

While historical chatbots like Eugene Goostman (which convinced some judges in 2014 by mimicking a thirteen-year-old boy) achieved brief Turing Test milestones, modern AI research rarely focuses on fooling human observers. Emulating human behavior includes mimicking human mistakes, biases, and biological processing limits. As the common historical analogy observes, aeronautical engineers did not successfully build commercial aircraft by copying bird feathers and flapping wings; they studied the underlying physics of aerodynamics and engine propulsion. Similarly, modern AI focuses on foundational principles of rationality, optimization, and search.

Recommended Video: AI History, Paradigms and AI Winters (YouTube Search Query: `History of AI Dartmouth to Deep Learning Stanford`).

4. System Taxonomies and Core Methodologies

Weak AI vs Strong AI

Weak (Narrow) AI: Engineering machines that act as if they were intelligent to solve specific tasks (e.g., Deep Blue, AlphaGo, modern speech recognition).

Strong (General) AI: Theoretical machines possessing genuine inner consciousness, authentic subjective experience, and generalized human-level intellect.

Symbolic vs Connectionist AI

Symbolic (Top-Down): Knowledge represented via explicit symbols, ontologies, and formal logic. Human-interpretable rules.

Connectionist (Bottom-Up): Sub-symbolic representation using artificial neural networks. Knowledge is encoded in distributed weights learned from data.

5. The Foundational Disciplines of Artificial Intelligence

AI is an interdisciplinary field shaped by several parent domains: **Philosophy** (epistemology, the mind-body problem, logic), **Mathematics** (formal proofs, algorithms, computational complexity, probability theory), **Economics** (rational decision theory, game theory, utility optimization), **Neuroscience** (biological neural networks and brain substrates), **Psychology** (cognitive modeling, perception, behavioral adaptation), **Computer Science** (data structures, hardware architectures, operating systems), and **Control Theory** (cybernetic feedback loops, dynamical systems, stability).

6. Historical Evolution: From Gestation to the Deep Learning Era

The technical gestation of AI began in 1943 when Warren McCulloch and Walter Pitts modeled artificial Boolean neurons. In 1951, Marvin Minsky and Dean Edmonds built the first operational neural network computer (SNARC). The field was formally christened at the 1956 Dartmouth Workshop organized by John McCarthy, Marvin Minsky, Nathaniel Rochester, and Claude Shannon.

The subsequent era of great expectations (1952 to 1969) produced Newell and Simon's Logic Theorist, McCarthy's LISP language, and early microworld problem solvers. However, early programs encountered combinatorial explosion when scaled beyond toy puzzles, triggering the first AI Winter (1966 to 1973). A major revival occurred in the 1970s and 1980s through knowledge-based Expert Systems (such as DENDRAL for chemical mass spectrometry, MYCIN for medical blood diagnostics, and R1/XCON at DEC, which saved forty million dollars annually).

WHY DID AI WINTERS OCCUR? KEY LESSONS FOR ENGINEERS

Early AI cycles collapsed because researchers underestimated the mathematical reality of combinatorial explosion and intractability (NP-hardness). Minsky and Papert proved in 1969 that single-layer perceptrons could not solve linearly non-separable problems like XOR. Progress revived in 1986 with backpropagation, and transformed in 2012 through massive web datasets, graphics processing units (GPUs), and deep neural architectures.

Recommended Video: Intelligent Agents and PEAS Framework (YouTube Search Query: `Intelligent Agents PEAS Framework Russell Norvig`).

7. The Core Concept of an Intelligent Agent

An **agent** is anything that perceives its external environment through physical or virtual **sensors** and acts upon that environment using **actuators** (or effectors). The abstract mathematical relation that assigns an action to any given percept sequence is known as the **agent function**: $f: P^* \rightarrow A$, where P^* denotes the history of all percepts received to date, and A is the set of possible actions. The computational engine executing this function is the **agent program**. Thus: *Agent = Physical Architecture + Program*.

8. Defining Rationality: Objective Success vs Omniscience

CRUCIAL DISTINCTIONS IN RATIONAL AGENT DESIGN

- **The Rational Standard:** For every possible percept sequence, a rational agent selects an action expected to maximize its performance measure, given the evidence provided by its percept history and built-in knowledge.
- **Rationality vs Omniscience:** An omniscient entity knows the actual future outcome of its actions with absolute certainty. A rational entity chooses the best choice given incomplete information. Rationality governs decision quality, not luck.
- **Information Gathering and Exploration:** Rational behavior mandates actively gathering missing knowledge (for example, looking both ways before crossing the road).
- **Autonomy:** An agent is autonomous if its actions are guided by its own experience, learning, and adaptation, rather than solely by rigid pre-programmed rules.

9. Systematic Agent Specification: The PEAS Framework

Before writing an algorithm, an engineer must rigorously define the problem specification across four essential pillars: **P**erformance measure, **E**nvironment, **A**ctuators, and **S**ensors.

Agent System	Performance Measure (P)	Environment (E)	Actuators (A)	Sensors (S)
Automated Taxi	Safety, speed, legal compliance, passenger comfort, profits	Roads, vehicle traffic, pedestrians, weather, customers	Steering wheel, accelerator, brake pedal, signals, horn, display	Cameras, lidar, radar, speedometer, GPS, accelerometer, engine sensors
Medical Diagnosis	Healthy patient, minimized healthcare costs, zero malpractice suits	Patient, clinical nursing staff, hospital facilities	Screen display of questions, laboratory tests, diagnoses, prescriptions	Keyboard data entry, digital symptom forms, laboratory measurement inputs
Part-Picking Robot	Percentage of parts placed in correct bins, speed, part integrity	Conveyor belt with mixed components, sorting bins	Jointed robotic manipulator arm, pneumatic suction gripper	Optical object camera, joint angle encoders, tactile grip pressure sensors
Interactive Tutor	Student test score improvement, topic retention, engagement rate	Set of students, curriculum syllabus, examination standards	Interactive display, generated practice problems, feedback explanations	Keyboard input, mouse interaction logs, timestamped student answers
Soccer Robot	Goals scored, goals prevented, ball possession time, fair play	Playing field, soccer ball, teammate robots, opponent robots	Motorized drive wheels, kicking solenoids, pivoting pan-tilt neck	Omnidirectional color camera, wheel encoders, wireless radio receiver

Recommended Video: Environment Classification Matrix (YouTube Search Query: `Environment Types Russell Norvig Artificial Intelligence`).

10. Environmental Properties and Classification Taxonomy

The structural nature of the environment dictates the algorithmic design of the agent. Russell and Norvig define seven primary dimensions of environmental classification:

- **Fully Observable vs Partially Observable:** An environment is fully observable if sensors provide complete access to the entire state of the environment at all times. Otherwise, parts remain hidden.
- **Deterministic vs Stochastic vs Strategic:** In deterministic worlds, the next state is uniquely fixed by the current state and the agent's action. In stochastic worlds, randomness or noise creates uncertainty. If the environment is deterministic except for the actions of other agents, it is termed strategic.
- **Episodic vs Sequential:** In episodic worlds, each task episode is atomic: the current action does not influence future episodes. In sequential worlds, current decisions have long-term consequences, requiring forward planning.
- **Static vs Dynamic vs Semidynamic:** A static environment does not change while the agent is deliberating. A dynamic environment continuously changes. If the world itself does not change with time but the agent's performance score does, it is semidynamic (for example, chess played with a ticking clock).
- **Discrete vs Continuous:** Discrete environments have a finite number of distinct states, percepts, and actions. Continuous environments involve infinite real-valued parameters (such as position, velocity, and steering angle).
- **Single-Agent vs Multi-Agent:** Distinguishes whether the agent operates alone, or interacts with other rational agents (which may be competitive, as in chess, or cooperative, as in shared traffic navigation).
- **Known vs Unknown:** Reflects the agent's knowledge of the underlying laws and physics governing the world, rather than sensor observability.

Environment Task	Observable?	Deterministic?	Episodic?	Static?	Discrete?	Agents
Image Classification	Fully	Deterministic	Episodic	Static	Continuous	Single
Solitaire Card Game	Partially	Deterministic	Sequential	Static	Discrete	Single
Chess with Clock	Fully	Strategic	Sequential	Semidynamic	Discrete	Multi (Competitive)
Backgammon (Dice)	Fully	Stochastic	Sequential	Static	Discrete	Multi (Competitive)
Customer Support Chat	Partially	Stochastic	Sequential	Dynamic	Discrete	Multi (Cooperative)
Autonomous Taxi Driving	Partially	Stochastic	Sequential	Dynamic	Continuous	Multi (Mixed)

PRACTICAL RULE: THE HARDEST ENVIRONMENT IN REAL-WORLD AI

The most challenging environment for an AI designer is **partially observable, stochastic, sequential, dynamic, continuous, and multi-agent**. Autonomous driving on public highways exhibits every one of these challenging attributes simultaneously.

Recommended Video: Agent Types and Subsumption Architecture (YouTube Search Query: `Five Agent Types Reflex Goal Utility Learning Russell`).

11. The Five Fundamental Agent Architectures

Agent programs range from basic lookup mappings to adaptive systems with internal memory, explicit goals, and multi-attribute utility evaluations:

1. Table-Driven & 2. Simple Reflex

Table-Driven: Direct lookup table indexed by percept histories. Fails due to astronomical table size (Chess > 10^{120}) and lack of adaptation.
Simple Reflex: Condition-action rules: `IF condition THEN action`. Stateless, lacks memory of past states. Trapped in infinite loops if percepts are ambiguous.

3. Model-Based (Internal Memory)

Maintains an internal state to track unobservable world features. Relies on two internal models: *How the world evolves independently*, and *How the agent's actions change the world*. Exemplified by Rodney Brooks' Subsumption Architecture (layered competence skills).

4. Goal-Based Agents

Combines internal world models with explicit descriptions of desirable goal states. Deliberative: evaluates candidate actions by simulating forward: *"What will happen if I take Action A?"* Incorporates search and automated planning algorithms.

5. Utility-Based Agents

Goals provide only a binary threshold (success or failure). Utility-based agents map world states to real numbers via a utility function: *U: State → Real Numbers*. Enables optimal trade-offs among conflicting goals and rational risk-balancing under uncertainty.

12. The Learning Agent Paradigm

Any of the above designs can be transformed into a **Learning Agent** by separating operational execution from adaptive improvement. A learning agent operates across four interactive modules:

- **Critic:** Evaluates the agent's behavior against an external, fixed performance standard. Provides objective evaluation feedback to the learning system.
- **Learning Element:** Takes feedback from the critic and updates the agent's internal decision rules to improve future performance.
- **Performance Element:** The operational agent core responsible for selecting external actions based on percepts and current rules.
- **Problem Generator:** Suggests novel, exploratory actions that lead to new experiences, preventing the agent from becoming trapped in suboptimal routines.

MEMORY TRICK: THE FIVE LEVELS OF COGNITIVE AUTONOMY

Remember the architectural hierarchy through a progression of human traits: **Simple Reflex** is biological instinct (pulling back from heat); **Model-Based** adds memory (remembering past sensory inputs); **Goal-Based** adds forward intent (charting a path to work); **Utility-Based** adds personal preferences (choosing comfort over speed); and **Learning Agent** adds education (analyzing errors to achieve mastery).

Recommended Video: Problem Formulation and Uninformed Search (YouTube Search Query: `Uninformed Search Algorithms BFS DFS UCS Russell Norvig`).

13. Problem Formulation for Search Agents

When an agent cannot immediately act reflexively, it must formulate a goal and explore alternate action sequences. A formal search problem is defined by five mathematical components:

1. **Initial State:** The starting situation the agent begins in (for example, *In(Arad)*).
2. **Actions:** The set of valid operators *Actions(s)* available in state *s*.
3. **Transition Model:** The mathematical function *Result(s, a)* returning the successor state produced by executing action *a* in state *s*.
4. **Goal Test:** A boolean test determining whether a given state satisfies the objective (explicit state matching or property testing).
5. **Path Cost:** A function assigning a numeric cost to a path, traditionally computed as the sum of step costs *c(s, a, s')*.

14. Performance Dimensions for Search Evaluation

Search algorithms are systematically assessed along four standardized theoretical criteria:

- **Completeness:** Is the algorithm guaranteed to find a valid solution if one exists?
- **Time Complexity:** How long (or how many generated nodes) does the search require to terminate?
- **Space Complexity:** How much computer memory is consumed by the search frontier and explored sets?
- **Optimality:** Does the strategy guarantee finding the lowest-cost path among all possible solutions?

Complexity bounds are expressed using three formal parameters: **b** (the maximum branching factor of the search tree), **d** (the depth of the shallowest optimal goal node), and **m** (the maximum depth of the state space, which may be infinite).

15. The Core Uninformed Search Algorithms

Uninformed (blind) search strategies use only the information supplied in the problem definition, traversing the graph systematically without heuristic guidance:

- **Breadth-First Search (BFS):** Expands the shallowest unexpanded node first using a First-In, First-Out (FIFO) queue. Guarantees finding the shallowest solution. However, storing every node in memory yields an exponential space complexity of $O(b^{d+1})$. Memory exhaustion occurs long before processor time runs out.
- **Uniform-Cost Search (UCS):** Expands the node with the lowest accumulated path cost *g(n)* using a priority queue. It is optimal for arbitrary non-negative step costs (where step cost $\geq \varepsilon > 0$). Equivalent to BFS when all step costs are identical.
- **Depth-First Search (DFS):** Expands the deepest unexpanded node first using a Last-In, First-Out (LIFO) stack. In graph search with loop detection, it achieves an exceptionally small memory footprint: $O(b \times m)$. However, it is neither complete in infinite trees nor optimal in path length.

TREE SEARCH VS GRAPH SEARCH: THE DANGER OF REPEATED STATES

In problems with reversible actions (such as grid mazes or sliding-tile puzzles), naive tree search will generate duplicate paths infinitely (A to B, B to A, A to B). Graph search eliminates this failure by maintaining an **explored set** (closed list). If a newly generated state has already been explored or is in the frontier with lower cost, it is discarded immediately.

Recommended Video: Iterative Deepening and Bidirectional Search (YouTube Search Query: `Iterative Deepening Search Proof and Complexity Russell`).

16. Depth-Limited and Iterative Deepening Search (IDS)

To prevent Depth-First Search from getting trapped in infinitely deep branches, **Depth-Limited Search (DLS)** imposes an explicit depth limit *l*. Any node at depth *l* is treated as having no children. While this guarantees termination, DLS is incomplete if the shallowest goal lies at depth $d > l$, and is suboptimal if $l > d$.

Iterative Deepening Search (IDS) solves this dilemma by iteratively calling Depth-Limited Search with gradually increasing limits: $l = 0, 1, 2, 3...$ until a goal is discovered. It terminates as soon as a goal is found at depth *d*.

17. Node Regeneration Overhead: Why IDS is Optimal

Students often assume that repeatedly generating upper nodes across successive iterations of IDS is wasteful. However, in an exponential tree with branching factor *b*, the vast majority of nodes reside in the deepest layer. The total nodes generated by IDS across all iterations is:

$$N(IDS) = (d + 1)(1) + (d)(b) + (d - 1)(b^2) + \ldots + (1)(b^d) = O(b^d)$$

For $b = 10$ and $d = 5$, BFS generates 111,111 nodes and consumes massive memory. IDS generates 123,456 total nodes: an overhead of barely 11%, while requiring memory for only $b \times d$ nodes (50 nodes). Hence, IDS is the preferred uninformed search method when the search space is large and solution depth is unknown.

18. Bidirectional Search and Complete Uninformed Comparison

Bidirectional Search simultaneously runs two searches: forward from the initial state, and backward from the goal state, stopping when the two frontiers intersect. This reduces time and space from $O(b^d)$ to $O(b^{d/2})$. However, it requires efficiently computing predecessors of the goal, which is difficult when multiple goal states exist or when goals are defined purely by abstract tests.

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional
Complete?	Yes (if b is finite)	Yes (if step cost ≥ ε)	No (loops in trees)	No (if l < d)	Yes (if b is finite)	Yes (if b finite & BFS)
Time	$O(b^{d+1})$	$O(b^{1 + \lfloor C^*/\varepsilon \rfloor})$	$O(b^m)$	$O(b^l)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^{d+1})$	$O(b^{1 + \lfloor C^*/\varepsilon \rfloor})$	O(b × m) [Linear]	O(b × l) [Linear]	O(b × d) [Linear]	$O(b^{d/2})$
Optimal?	Yes (if costs equal)	Yes (optimal cost)	No	No	Yes (if costs equal)	Yes (if costs equal)

RULE OF THUMB: WHEN TO CHOOSE EACH SEARCH METHOD

Use **BFS** only when solution paths are shallow and memory is abundant. Use **UCS** when actions have differing non-negative costs. Use **DFS** when memory is severely constrained and solutions are abundant. Use **IDS** for general uninformed tree and graph search problems with uniform step costs.

Recommended Video: Informed Search and Greedy Best-First Search (YouTube Search Query: `Greedy Best First Search Romania Example Russell Norvig`).

19. The Concept of Heuristic Informed Search

While uninformed strategies explore blindly, **informed (heuristic) search** incorporates domain-specific knowledge beyond the problem definition. This is achieved via an evaluation function *f(n)* that prioritizes nodes that appear most promising. Most evaluation functions incorporate a **heuristic function** *h(n)*, defined as the estimated cost of the cheapest path from node *n* to a goal state. Heuristics are non-negative, problem-specific functions constrained by the rule: *if n is a goal node, then h(n) = 0*.

20. Greedy Best-First Search (GBFS) Defined

Greedy Best-First Search expands the node that appears closest to the goal, evaluating nodes solely through their heuristic estimate:

```
f(n) = h(n)
```

In route planning across Romania, the Straight-Line Distance heuristic $h_{SLD}(n)$ gives the straight Euclidean distance from any city to the destination (Bucharest). GBFS uses this distance to greedily pick the next city.

21. Step-by-Step Trace of GBFS on the Romania Network

Let our initial state be **Arad** (h_{SLD} = 366), with goal **Bucharest** (h_{SLD} = 0):

1. **Expand Arad:** Successors are Sibiu (h = 253), Timisoara (h = 329), and Zerind (h = 374). GBFS greedily selects **Sibiu** because its heuristic value (253) is the lowest.
2. **Expand Sibiu:** Successors are Arad (366), Fagaras (h = 176), Oradea (h = 380), and Rimnicu Vilcea (h = 193). The lowest heuristic is **Fagaras** (176), so GBFS selects Fagaras.
3. **Expand Fagaras:** Successors are Sibiu (253) and Bucharest (h = 0). GBFS immediately selects **Bucharest**.

The Path Found: Arad → Sibiu → Fagaras → Bucharest. Total actual distance travelled: *140 + 99 + 211 = 450 km*.

> **WHY GREEDY BEST-FIRST SEARCH IS SUBOPTIMAL**
>
> GBFS found a path costing 450 km. However, the optimal path through the road network travels via **Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest**, costing *140 + 80 + 97 + 101 = 418 km*. GBFS failed to discover this better path because Fagaras appeared closer to Bucharest in straight-line distance (176 km) than Rimnicu Vilcea (193 km). Like a hiker walking straight toward a visible mountain peak without seeing the canyon in between, GBFS is short-sighted: it minimizes estimated future cost while ignoring accumulated past cost.

22. Theoretical Properties of GBFS

Completeness: Incomplete in tree search (can become trapped in infinite loops). Complete in finite state spaces with repeated-state checking (graph search).

Time and Space: Worst-case time is $O(b^m)$ and space is $O(b^m)$ because all generated nodes are retained in memory. A high-quality heuristic dramatically reduces practical run times.

Recommended Video: Manhattan Distance and 8-Puzzle Calculation (YouTube Search Query: `Manhattan Distance 8 Puzzle Heuristic Calculation Artificial Intelligence`).

23. Deep Dive: Manhattan Distance Metric in Grid Domains

In sliding-tile puzzles and grid mazes, diagonal steps are prohibited. An object must move along orthogonal grid lines (up, down, left, right). The **Manhattan Distance** (also called Taxicab or L_1 distance) between two grid points (x_1, y_1) and (x_2, y_2) is the sum of horizontal and vertical absolute coordinate differences:

$$\text{Manhattan Distance} = |x_1 - x_2| + |y_1 - y_2|$$

24. Complete Step-by-Step 8-Puzzle Manhattan Calculation

Consider the exact 8-puzzle configurations from the lecture slides, indexed as (row, column) from (0, 0) to (2, 2):

Start State

7	2	4
5		6
8	3	1

→ **Goal Target** →

Goal State

	1	2
3	4	5
6	7	8

Tile	Start Position (r, c)	Goal Position (r, c)	Row Offset $\lvert r_{start} - r_{goal} \rvert$	Col Offset $\lvert c_{start} - c_{goal} \rvert$	Individual Manhattan Distance
Tile 1	Row 2, Col 2	Row 0, Col 1	$\lvert 2 - 0 \rvert = 2$	$\lvert 2 - 1 \rvert = 1$	2 + 1 = **3**
Tile 2	Row 0, Col 1	Row 0, Col 2	$\lvert 0 - 0 \rvert = 0$	$\lvert 1 - 2 \rvert = 1$	0 + 1 = **1**
Tile 3	Row 2, Col 1	Row 1, Col 0	$\lvert 2 - 1 \rvert = 1$	$\lvert 1 - 0 \rvert = 1$	1 + 1 = **2**
Tile 4	Row 0, Col 2	Row 1, Col 1	$\lvert 0 - 1 \rvert = 1$	$\lvert 2 - 1 \rvert = 1$	1 + 1 = **2**
Tile 5	Row 1, Col 0	Row 1, Col 2	$\lvert 1 - 1 \rvert = 0$	$\lvert 0 - 2 \rvert = 2$	0 + 2 = **2**
Tile 6	Row 1, Col 2	Row 2, Col 0	$\lvert 1 - 2 \rvert = 1$	$\lvert 2 - 0 \rvert = 2$	1 + 2 = **3**
Tile 7	Row 0, Col 0	Row 2, Col 1	$\lvert 0 - 2 \rvert = 2$	$\lvert 0 - 1 \rvert = 1$	2 + 1 = **3**
Tile 8	Row 2, Col 0	Row 2, Col 2	$\lvert 2 - 2 \rvert = 0$	$\lvert 0 - 2 \rvert = 2$	0 + 2 = **2**

Total Puzzle Manhattan Distance h_2(Start) = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18.

25. How Greedy Best-First Search Selects the Next Move

From this start configuration, the blank tile at center (1, 1) can swap with four neighbors: Tile 2 (down), Tile 3 (up), Tile 5 (right), or Tile 6 (left). If the blank swaps with Tile 3 (moving Tile 3 from (2, 1) into (1, 1)), Tile 3's offset to its goal (1, 0) becomes $|1 - 1| + |1 - 0| = 1$. This reduces Tile 3's distance from 2 to 1, lowering the total puzzle Manhattan heuristic from 18 to 17. Because 17 is strictly lower than any other candidate action, GBFS greedily expands this move.

EXAMINATION STEP-BY-STEP CHECKLIST FOR MANHATTAN DISTANCE

1. List every numbered tile from 1 to 8 (never calculate distance for the empty blank space).
2. Identify the current grid coordinates (row, col) and goal coordinates (row, col) for each tile.
3. Compute absolute row difference plus absolute column difference for each tile.
4. Sum all eight individual tile values together to produce the final puzzle heuristic score h_2.

Recommended Video: A* Search Algorithm and Optimality Proof (YouTube Search Query: `A Star Search Algorithm Admissibility Consistency Proof Russell`).

26. The A* Search Paradigm

While Greedy Best-First minimizes estimated future cost *h(n)* and Uniform-Cost minimizes past accumulated cost *g(n)*, **A* Search** combines both perspectives into a single unified evaluation function:

```
f(n) = g(n) + h(n)
```

Here, *g(n)* is the actual cost incurred from the start state to node *n*, and *h(n)* is the estimated cost from *n* to the goal. Thus, *f(n)* represents the estimated total cost of the cheapest complete solution passing through node *n*.

27. Step-by-Step Trace of A* on the Romania Network

Starting at **Arad**, where *f(Arad) = 0 + 366 = 366*:

1. **Expand Arad:** Successors are Sibiu (f = 140 + 253 = 393), Timisoara (f = 118 + 329 = 447), Zerind (f = 75 + 374 = 449). A* expands **Sibiu** (f = 393).
2. **Expand Sibiu:** Successors are Fagaras (f = 239 + 176 = 415), Rimnicu Vilcea (f = 220 + 193 = 413), Oradea (f = 291 + 380 = 671), Arad (f = 280 + 366 = 646). A* expands **Rimnicu Vilcea** (f = 413).
3. **Expand Rimnicu Vilcea:** Successors are Pitesti (f = 317 + 100 = 417), Craiova (f = 366 + 160 = 526), Sibiu (f = 300 + 253 = 553). The current lowest node in the frontier is now **Fagaras** (f = 415).
4. **Expand Fagaras:** Successors are Bucharest (f = 450 + 0 = 450) and Sibiu (591). Now **Pitesti** (f = 417) is lowest in the frontier.
5. **Expand Pitesti:** Successors are Bucharest (f = 418 + 0 = 418), Craiova (615), Rimnicu Vilcea (607).
6. **Goal Selection:** Bucharest at **f = 418** is now the lowest node on the frontier. A* selects it and returns the optimal path: *Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest (418 km)*.

28. Conditions for Optimality: Admissibility and Consistency

Admissibility: An admissible heuristic never overestimates the true cost to reach the goal: $h(n) \leq C^*(n)$ for all nodes, with $h(Goal) = 0$. It is optimistic. Straight-line distance is admissible because a straight line is the shortest possible path between two points.

Consistency (Monotonicity): A heuristic is consistent if for every node *n* and successor *n′* reached via action *a*:

```
h(n) ≤ c(n, a, n') + h(n')
```

MATHEMATICAL PROOF: WHY CONSISTENT A* IS GUARANTEED TO BE OPTIMAL

By definition: $g(n') = g(n) + c(n, a, n')$. Adding $h(n')$ gives: $f(n') = g(n') + h(n') = g(n) + c(n, a, n') + h(n') \geq g(n) + h(n) = f(n)$. Therefore, **f(n) is monotonically non-decreasing along any path**. A* expands nodes in concentric contour bands of increasing *f* values. Because $h(Goal) = 0$, the *f* value of any goal node equals its true cost. Hence, the first goal node selected for expansion must be optimal.

Recommended Video: Heuristic Dominance, RBFS and Local Optimization (YouTube Search Query: `Effective Branching Factor Heuristic Dominance RBFS Russell`).

29. Evaluating Heuristic Quality and Effective Branching Factor

The quality of a heuristic is measured by its **effective branching factor (b*)**. If A* generates *N* total nodes to find a solution at depth *d*, then *b** is the branching factor of a uniform tree of depth *d* containing *N + 1* nodes:

$$N + 1 = 1 + b^* + (b^*)^2 + \ldots + (b^*)^d$$

For example, if A* finds a solution at depth d = 5 using N = 52 nodes, then b* = 1.92. A well-designed heuristic achieves an effective branching factor close to 1.

30. Heuristic Dominance and Problem Relaxation

If two heuristics h_1 and h_2 are both admissible, and $\mathbf{h_2(n) \geq h_1(n)}$ for all nodes *n*, then h_2 *dominates* h_1. A* using h_2 is mathematically guaranteed to expand fewer or equal nodes than A* using h_1.

Admissible heuristics are formally generated from **relaxed problems** (problems with fewer constraints). In the 8-puzzle:

- If a tile can move anywhere directly: we derive h_1 (Misplaced Tiles).
- If a tile can move to any adjacent square overlapping others: we derive h_2 (Manhattan Distance).
- Because every valid move in h_2 includes h_1 moves plus adjacency restrictions, $h_2(n) \geq h_1(n)$ always. Thus Manhattan Distance dominates Misplaced Tiles.
- If multiple admissible heuristics are available: $h(n) = max(h_a(n), h_b(n))$ is always admissible and dominates each individual heuristic.

31. Memory-Bounded Search: Recursive Best-First Search (RBFS)

A* stores all generated nodes in memory, causing failure on massive search spaces. **Recursive Best-First Search (RBFS)** mimics standard best-first search within linear space: $O(b \times d)$. It executes recursively like DFS, maintaining an `f_limit` representing the lowest *f*-value of the best alternative path available to its ancestors. When the current node's *f*-value exceeds this limit, recursion unwinds back to the alternative path, updating the forgotten branch with the backed-up best *f*-value of its children.

LOCAL SEARCH OPTIMIZATION: THE LANDSCAPE METAPHOR

When the path to the solution does not matter and only the final configuration is required (as in the 8-Queens problem, Traveling Salesperson, or integrated circuit VLSI layout), algorithms operate via **local search**. The state space is modeled as an elevation surface:

- **Global Maximum/Minimum:** The absolute best configuration across the entire state space.
- **Local Maximum:** A state higher than all its immediate neighbors, but lower than the global peak.
- **Plateau (Flat Local Maximum):** An area where all adjacent states share the same evaluation score.
- **Shoulder:** A flat plateau that subsequently leads uphill to higher peaks.

> Recommended Video: Minimax and Alpha-Beta Pruning (YouTube Search Query: `Minimax Algorithm Alpha Beta Pruning Example MIT Winston`).

32. Local Search Algorithms and Evolutionary Methods

- **Hill-Climbing Search (Steepest Ascent):** Greedily steps to the neighbor with the highest value. Analogized to "climbing Everest in thick fog with amnesia." Susceptible to becoming trapped on local maxima, ridges, and plateaus. Overcome via random restarts, lookahead, or stochastic noise.
- **Simulated Annealing:** Inspired by the physical annealing of molten metals. It randomly selects a neighbor. If the neighbor improves value ($\Delta E > 0$), it is accepted. If worse ($\Delta E < 0$), it is accepted probabilistically using the Boltzmann factor: $P = e^{\Delta E / T}$. At high temperature T, downhill moves are freely taken to escape local maxima. As T cools to zero, it converges to greedy hill-climbing.
- **Genetic Algorithms (GA):** Stochastic local beam search where successors are generated by combining pairs of states (chromosomes). Driven by three operators: *Selection* (probability proportional to fitness), *Crossover* (recombining genetic substrings from two parents), and *Mutation* (random bit/gene flips).

33. Adversarial Search and Game Formulations

Games are formal competitive environments where an agent faces an opponent whose decisions cannot be controlled. A deterministic, zero-sum two-player game is defined by an **Initial State**, legal **Actions(s)**, a **Transition Model Result(s, a)**, a **Terminal Test**, and a **Utility Function** (e.g., +1 for win, 0 for draw, -1 for loss).

34. The Minimax Algorithm and Alpha-Beta Pruning

Assuming a rational opponent who minimizes our payoff, the Minimax value represents the best achievable payoff:

$$\text{Minimax}(s) = \begin{cases} \text{Utility}(s) & \text{if Terminal}(s); \\ \max_a \text{Minimax}(\text{Result}(s, a)) & \text{for MAX}; \\ \min_a \text{Minimax}(\text{Result}(s, a)) & \text{for MIN} \end{cases}$$

Minimax has time complexity $O(b^m)$ and space $O(bm)$. In real games (Chess has $b \approx 35$, $m \approx 80$), complete tree evaluation is impossible. **Alpha-Beta Pruning** eliminates branches that cannot influence the final decision:

- **α (Alpha):** The value of the best (highest) choice found so far along the path for MAX (initialized to -∞).
- **β (Beta):** The value of the best (lowest) choice found so far along the path for MIN (initialized to +∞).
- **Pruning Criterion:** If at any point **α ≥ β**, the remaining children of that node are pruned immediately. With optimal move ordering, time complexity drops to $O(b^{m/2})$, doubling the searchable search depth.

> **REAL-TIME GAME PLAY: HEURISTIC CUTOFFS AND QUIESCENCE SEARCH**
>
> Under strict tournament time clocks, programs stop search at depth d and evaluate states using a heuristic evaluation function: $Eval(s) = w_1 f_1(s) + w_2 f_2(s) + \ldots + w_k f_k(s)$, weighing piece values and positional safety. To prevent blunders caused by stopping search in the middle of a piece exchange (the horizon problem), programs apply **quiescence search**, expanding volatile capture moves deeper until a stable position is reached.